Logistics Optimization for Delivery Routes – Amazon

Project Overview: Amazon, a global e-commerce leader, handles millions of daily shipments across various regions. The logistics network is vast, comprising warehouses, fulfillment centers, and last-mile delivery partners. With increasing order volumes, delays and inefficiencies in delivery routes can significantly affect customer satisfaction and operational costs.

- Identify and delete duplicate Order_ID records.

```
SELECT    Order_ID,    COUNT(Order_ID) AS Duplicate_orderID  FROM
    ordersGROUP BY    Order_IDHAVING    COUNT(Order_ID) > 1
                                  LIMIT 0, 1000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Duplicate_orderID

```
-- Replace null Traffic_Delay_Min with the average delay for that route
select avg(Traffic_Delay_Min) as avg_Traffic_Delay_Minfrom routes;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg_Traffic_Delay_Min
28.8500

-- Convert all date columns into YYYY-MM-DD format using SQL functions.

```
select Order_ID ,date_format(Order_Date,'%y-%m-%d') as new_order_date,date_format(Expected_Delivery_Date,'%y-%m-%d') as new_Expected_Delivery_Date,date_format(Actual_Delivery_Date,'%y-%m-%d') as new_Actual_Delivery_Datefrom orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	new_order_date	new_Expected_Delivery_Date	new_Actual_Delivery_Date
O0001	25-06-08	25-06-11	25-06-11
O0002	25-06-22	25-06-27	25-06-29
O0003	25-06-04	25-06-09	25-06-11
O0004	25-06-28	25-07-05	25-07-07
O0005	25-06-04	25-06-09	25-06-10
O0006	25-07-17	25-07-19	25-07-19
O0007	25-06-14	25-06-17	25-06-18
O0008	25-06-23	25-06-30	25-07-02
O0009	25-07-11	25-07-16	25-07-18
O0010	25-06-14	25-06-21	25-06-21
O0011	25-05-26	25-06-02	25-06-02
O0012	25-07-16	25-07-20	25-07-21

orders 3 | Result 4

```
-- Average Delivery Delay per Region (Start_Location).
SELECT    R.Start_Location AS Region,    AVG(DATEDIFF(O.Actual_Delivery_Date, O.Expected_Delivery_Date)) AS
AvgDeliveryDelayDaysFROM    Orders OINNER JOIN    routes R ON O.Route_ID = R.Route_ID WHERE    O.Actual_Delivery_Date
IS NOT NULL    AND O.Expected_Delivery_Date IS NOT NULL    AND DATEDIFF(O.Actual_Delivery_Date,
O.Expected_Delivery_Date) > 0GROUP BY    R.Start_LocationORDER BY    AvgDeliveryDelayDays DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Region	AvgDeliveryDelayDays
South Jenniferfurt	2.0000
Lake Oscarfurt	2.0000
Port Tammybury	1.7500
New Jessica	1.7143
Lake Kathrynchester	1.6667
New Aimeechester	1.6667
West Calvinview	1.6250
Ramosside	1.6000
North Edward	1.6000
South Brianchester	1.5714
Aaronhaven	1.5714
Lowemouth	1.5556

Result 1

```
-- On-Time Delivery % = (Total On-Time Deliveries / Total Deliveries) * 100.
SELECT    COUNT(Order_ID) AS TotalDeliveries,    SUM(CASE        WHEN DATEDIFF(Actual_Delivery_Date, Expected_Delivery_Date) <= 0 THEN 1
ELSE 0    END) AS TotalOnTimeDeliveries,    (SUM(CASE        WHEN DATEDIFF(Actual_Delivery_Date,Expected_Delivery_Date) <= 0 THEN 1
ELSE 0    END) / COUNT(Order_ID)) * 100 AS OnTimeDeliveryPercentageFROM    OrdersWHERE    Actual_Delivery_Date IS NOT NULL     AND
Expected_Delivery_Date IS NOT NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	TotalDeliveries	TotalOnTimeDeliveries	OnTimeDeliveryPercentage
▶	300	168	56.0000

-- Average Traffic Delay per Route.
select Route_ID, Start_Location, avg(Traffic_Delay_Min) as AvgTraffic_delay from routes where Traffic_Delay_Min is not null group by Route_ID, Start_Location order by AvgTraffic_delay desc;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Route_ID	Start_Location	AvgTraffic_delay
R014	North Edward	58.0000
R007	Aaronhaven	51.0000
R009	Bellfurt	46.0000
R006	Lake Robinhaven	42.0000
R010	North Aliceside	40.0000
R013	South Johnville	38.0000
R019	Lake Oscarfurt	37.0000
R001	New Aimeechester	36.0000
R008	South Davidborough	34.0000
R012	West Calvinview	29.0000
R015	South Jenniferfurt	29.0000
R005	New Jessica	26.0000

```
-- Ensure that no Actual_Delivery_Date is before Order_Date (flag such records).
SELECT Order_ID, Order_Date, Actual_Delivery_Date, CASE WHEN Actual_Delivery_Date < Order_Date THEN 'FLAG: Delivery Before Order' ELSE 'Valid' END AS Delivery_Date_CheckFROM orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Order_Date	Actual_Delivery_Date	Delivery_Date_Check
O0289	2025-05-26 00:00:00	2025-05-29 00:00:00	Valid
O0290	2025-05-29 00:00:00	2025-06-04 00:00:00	Valid
O0291	2025-06-07 00:00:00	2025-06-13 00:00:00	Valid
O0292	2025-06-28 00:00:00	2025-07-02 00:00:00	Valid
O0293	2025-06-24 00:00:00	2025-06-30 00:00:00	Valid
O0294	2025-07-08 00:00:00	2025-07-16 00:00:00	Valid
O0295	2025-05-22 00:00:00	2025-05-27 00:00:00	Valid
O0296	2025-06-12 00:00:00	2025-06-19 00:00:00	Valid
O0297	2025-06-15 00:00:00	2025-06-21 00:00:00	Valid
O0298	2025-07-20 00:00:00	2025-07-27 00:00:00	Valid
O0299	2025-05-30 00:00:00	2025-06-03 00:00:00	Valid
O0300	2025-05-29 00:00:00	2025-06-03 00:00:00	Valid

Result 1

-- Calculate delivery delay (in days) for each order
select Order_ID , Expected_Delivery_Date,Actual_Delivery_Date , datediff(Actual_Delivery_Date,Expected_Delivery_Date) as Delayed_Delivey_daysfrom orders;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Expected_Delivery_Date	Actual_Delivery_Date	Delayed_Delivey_days
O0001	2025-06-11 00:00:00	2025-06-11 00:00:00	0
O0002	2025-06-27 00:00:00	2025-06-29 00:00:00	2
O0003	2025-06-09 00:00:00	2025-06-11 00:00:00	2
O0004	2025-07-05 00:00:00	2025-07-07 00:00:00	2
O0005	2025-06-09 00:00:00	2025-06-10 00:00:00	1
O0006	2025-07-19 00:00:00	2025-07-19 00:00:00	0
O0007	2025-06-17 00:00:00	2025-06-18 00:00:00	1
O0008	2025-06-30 00:00:00	2025-07-02 00:00:00	2
O0009	2025-07-16 00:00:00	2025-07-18 00:00:00	2
O0010	2025-06-21 00:00:00	2025-06-21 00:00:00	0
O0011	2025-06-02 00:00:00	2025-06-02 00:00:00	0
O0012	2025-07-20 00:00:00	2025-07-21 00:00:00	1

Result 2

-- Find Top 10 delayed routes based on average delay days.

```
SELECT    Route_ID,    AVG(DATEDIFF( Expected_Delivery_Date, Actual_Delivery_Date)) AS
Avg_delayed_daysFROM    ordersGROUP BY    Route_IDORDER BY    Avg_delayed_days DESCLIMIT 10;
```

Result Grid | Filter Rows:

Route_ID	Avg_delayed_days
R016	-0.4167
R008	-0.4375
R018	-0.4667
R002	-0.5000
R003	-0.5000
R009	-0.5000
R006	-0.5455
R014	-0.6154
R019	-0.6154
R001	-0.6667

```
-- Use window functions to rank all orders by delay within each warehouse.
SELECT    Order_ID,    Warehouse_ID,    DATEDIFF( Actual_Delivery_Date, Expected_Delivery_Date) AS delay_time_days,
dense_rank() OVER (        PARTITION BY warehouse_id        ORDER BY DATEDIFF( Actual_Delivery_Date,
Expected_Delivery_Date) DESC    ) AS warehouse_delay_rank FROM    Orders ORDER BY    Warehouse_ID,
warehouse_delay_rank;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Warehouse_ID	delay_time_days	warehouse_delay_rank
O0135	W007	0	3
O0096	W007	0	3
O0129	W007	0	3
O0172	W007	0	3
O0067	W007	0	3
O0200	W008	2	1
O0003	W008	2	1
O0033	W008	2	1
O0231	W008	2	1
O0023	W008	1	2
O0243	W008	0	3
O0143	W008	0	3

Result 1

each route, calculate: -- Average delivery time (in days). -- Average traffic delay. -- Distance-to-time efficiency ratio: Distance_KM / Average_Travel_Time_Min

select Route_ID, avg(datediff(Actual_Delivery_Date,Order_Date)) as average_delivery_days from orders group by Route_ID order by Route_ID;

Result Grid Filter Rows: Exp

Route_ID	average_delivery_days
R001	5.0000
R002	4.7273
R003	4.5000
R004	5.1000
R005	5.0000
R006	4.8182
R007	5.7500
R008	5.4375
R009	4.8750
R010	5.2500
R011	5.5500
R012	4.8333

Result 4

Average traffic delay.

```
select Route_ID , avg(Traffic_Delay_Min) as avg_traffic_delaY_min from routes group by Route_ID order by Route_ID;
```

Result Grid | Filter Rows:

Route_ID	avg_traffic_delaY_min
R001	36.0000
R002	22.0000
R003	11.0000
R004	3.0000
R005	26.0000
R006	42.0000
R007	51.0000
R008	34.0000
R009	46.0000
R010	40.0000
R011	25.0000
R012	29.0000

Result 5

-- Distance-to-time efficiency ratio: Distance_KM / Average_Travel_Time_Min.

```
select Route_ID,  round(Sum(Distance_KM) * 1 /sum(Average_Travel_Time_Min),2) as efficiency_ratio from routes group by Route_ID order by Route_ID;
```

Result Grid | Filter Rows:

Route_ID	efficiency_ratio
R001	0.61
R002	2.16
R003	0.58
R004	4.28
R005	0.37
R006	0.16
R007	0.28
R008	0.24
R009	0.6
R010	0.67
R011	0.96
R012	0.9

Result 6

-- Identify 3 routes with the worst efficiency ratio.

```
select Route_ID,  round(Sum(Distance_KM) * 1 /sum(Average_Travel_Time_Min),2) as efficiency_ratio from routes group by Route_ID order by efficiency_ratio asc limit 3;
```

Result Grid | Filter Rows:

Route_ID	efficiency_ratio
R016	0.15
R006	0.16
R008	0.24

-- Find routes with >20% delayed shipments.
-- Recommend potential routes for optimization.

```
SELECT    route_id,    COUNT(*) AS total_shipments,    SUM(CASE WHEN Delivery_Status = 'Delayed' THEN 1 ELSE 0 END) AS delayed_shipments,
(CAST(SUM(CASE WHEN Delivery_Status = 'Delayed' THEN 1 ELSE 0 END) AS REAL) * 100.0 / COUNT(*)) AS percentage_delayedFROM
ordersGROUP BY    route_idHAVING    (CAST(SUM(CASE WHEN Delivery_Status = 'Delayed' THEN 1 ELSE 0 END) AS REAL) * 100.0 / COUNT(*)) >
20.0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

route_id	total_shipments	delayed_shipments	percentage_delayed
R005	15	7	46.666666666666664
R004	20	10	50
R013	17	9	52.94117647058823
R012	12	8	66.66666666666667
R002	22	7	31.818181818181817
R020	17	12	70.58823529411765
R008	16	5	31.25
R011	20	9	45
R017	14	8	57.142857142857146
R014	13	5	38.46153846153846
R007	16	7	43.75
R006	11	4	36.36363636363637

Result 1

-- Find the top 3 warehouses with the highest average processing time.
Select Warehouse_ID, avg(Processing_Time_Min)as Avg_Processing_time from warehouses Group by Warehouse_ID order by Warehouse_ID desc limit 3;

Result Grid | Filter Rows: | Export: | Wr

Warehouse_ID	Avg_Processing_time
W010	171.0000
W009	170.0000
W008	161.0000

```
-- Calculate total vs. delayed shipments for each warehouse.
SELECT    O.Warehouse_ID,    COUNT(O.Order_ID) AS total_shipments,    SUM(CASE WHEN O.Delivery_Status =
'Delayed' THEN 1 ELSE 0 END) AS delayed_shipmentsFROM    orders OJOIN   warehouses W  ON
O.Warehouse_ID = W.Warehouse_IDGROUP BY    O.Warehouse_IDORDER BY    delayed_shipments DESC;
```

Result Grid | Filter Rows: | Export: | Wra

Warehouse_ID	total_shipments	delayed_shipments
W010	30	20
W001	34	18
W007	38	15
W005	25	14
W009	33	13
W003	23	13
W004	31	12
W006	37	12
W002	28	10
W008	21	5

```
-- Use CTEs to find bottleneck warehouses where processing time > global average.
WITH GlobalAverage AS (    SELECT        AVG(Processing_Time_Min) AS avg_time    FROM        Warehouses),WarehouseAverages
AS (    SELECT        Warehouse_ID,        AVG(Processing_Time_Min) AS warehouse_avg_processingtime    FROM        Warehouses
GROUP BY        Warehouse_ID)SELECT    wa. Warehouse_ID,    warehouse_avg_processingtime,    ga.avg_time AS
global_average_time,        warehouse_avg_processingtime - ga.avg_time AS time_differenceFROM    WarehouseAverages
waCROSS JOIN    GlobalAverage ga WHERE    warehouse_avg_processingtime > ga.avg_timeORDER BY    time_difference DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Warehouse_ID	warehouse_avg_processingtime	global_average_time	time_difference
W010	171.0000	110.1000	60.9000
W009	170.0000	110.1000	59.9000
W007	166.0000	110.1000	55.9000
W008	161.0000	110.1000	50.9000
W001	142.0000	110.1000	31.9000

```
-- Rank warehouses based on on-time delivery percentage.
 WITH WarehousePerformance AS (    SELECT        Warehouse_ID,              CAST(SUM(CASE WHEN Delivery_Status = 'On Time'
THEN 1 ELSE 0 END) AS DECIMAL) AS OnTimeDeliveries,              COUNT(*) AS TotalDeliveries,              (CAST(SUM(CASE
WHEN Delivery_Status = 'On Time' THEN 1 ELSE 0 END) AS DECIMAL) * 100.0) / COUNT(*) AS OTD_Percentage    FROM
orders    GROUP BY        Warehouse_ID)SELECT    Warehouse_ID,    OTD_Percentage,    TotalDeliveries,        DENSE_RANK() OVER
(ORDER BY OTD_Percentage DESC) AS Warehouse_RankFROM    WarehousePerformanceORDER BY    Warehouse_Rank ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Warehouse_ID	OTD_Percentage	TotalDeliveries	Warehouse_Rank
W008	76.19048	21	1
W006	67.56757	37	2
W002	64.28571	28	3
W004	61.29032	31	4
W009	60.60606	33	5
W007	60.52632	38	6
W001	47.05882	34	7
W005	44.00000	25	8
W003	43.47826	23	9
W010	33.33333	30	10

Result 1

-- Rank agents (per route) by on-time delivery percentage

```
Select Route_ID,Agent_ID,On_Time_Percentage,dense_rank () over ( partition by Route_ID order by On_Time_Percentage desc) as Rankingfrom deliveryagents order by Route_ID,Ranking;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Route_ID	Agent_ID	On_Time_Percentage	Ranking
R001	A006	81.56	1
R001	A037	78.94	2
R002	A033	98.01	1
R002	A024	97.16	2
R002	A027	73.8	3
R002	A050	72.75	4
R002	A011	71.5	5
R003	A015	83.31	1
R005	A029	81.71	1
R005	A035	74.5	2
R006	A041	91.47	1
R006	A047	84.62	2

Result 1

Output

-- Find agents with on-time % < 80%.
SELECT Route_ID, Agent_ID, On_Time_PercentageFROM deliveryagents WHERE On_Time_Percentage < 80ORDER BY Route_ID, On_Time_Percentage;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Route_ID	Agent_ID	On_Time_Percentage
R007	A040	77.83
R012	A048	77.92
R012	A016	78.39
R014	A003	74.15
R014	A030	76.6
R016	A039	72.6
R016	A043	79.51
R017	A023	74.21
R018	A038	71.8
R018	A008	72.39
R019	A010	70.45
R020	A046	75.46

liveryagents 1

```
-- Compare average speed of top 5 vs bottom 5 agents using subqueries.
WITH RankedAgents AS (    SELECT        Agent_ID,       Avg_Speed_KM_HR,       On_Time_Percentage,        DENSE_RANK() OVER (ORDER BY
On_Time_Percentage DESC) AS Rank_Top,        DENSE_RANK() OVER (ORDER BY On_Time_Percentage ASC) AS Rank_Bottom    FROM
deliveryagents),Top5Speeds AS (    SELECT        AVG(Avg_Speed_KM_HR) AS Avg_Speed_Top5    FROM        RankedAgents    WHERE        Rank_Top
<= 5),Bottom5Speeds AS (        SELECT        AVG(Avg_Speed_KM_HR) AS Avg_Speed_Bottom5    FROM        RankedAgents    WHERE
Rank_Bottom <= 5)SELECT    t5.Avg_Speed_Top5,    b5.Avg_Speed_Bottom5FROM    Top5Speeds t5CROSS JOIN    Bottom5Speeds b5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Avg_Speed_Top5	Avg_Speed_Bottom5
60.0000	56.8000

-- For each order, list the last checkpoint and time.

```
WITH RankedCheckpoints AS (
    SELECT
        Shipment_ID,
        Order_ID,
        Checkpoint_Time,
        Checkpoint,
        ROW_NUMBER() OVER (
PARTITION BY Shipment_ID
        ORDER BY Checkpoint_Time DESC
        ) as rn
    FROM
        shipmenttrackingtable )SELECT
        Shipment_ID,
Order_ID,
    Checkpoint_Time AS LastUpdateTime,
    Checkpoint AS LastLocationFROM
    RankedCheckpointsWHERE
    rn = 1;
```

Shipment_ID	Order_ID	LastUpdateTime	LastLocation
SO00011	O0001	2025-06-09	Checkpoint 1
SO00012	O0001	2025-06-09	Checkpoint 2
SO00021	O0002	2025-06-22	Checkpoint 1
SO00022	O0002	2025-06-22	Checkpoint 2
SO00023	O0002	2025-06-23	Checkpoint 3
SO00024	O0002	2025-06-23	Checkpoint 4
SO00031	O0003	2025-06-04	Checkpoint 1
SO00032	O0003	2025-06-05	Checkpoint 2
SO00033	O0003	2025-06-05	Checkpoint 3
SO00034	O0003	2025-06-05	Checkpoint 4
SO00041	O0004	2025-06-28	Checkpoint 1
SO00042	O0004	2025-06-29	Checkpoint 2

Result 1

-- Find the most common delay reasons (excluding None).

```
select Delay_Reason, count(Shipment_ID) as Numberofshipmentdelays from shipmenttrackingtable where Delay_Reason is not null and Delay_Reason != 'None' group by Delay_Reason order by Numberofshipmentdelays desc limit 25;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Delay_Reason	Numberofshipmentdelays
Sorting Delay	272
Weather	263
Traffic	247

```
-- Identify orders with >2 delayed checkpoints
WITH DelayedCheckpoints AS (
    SELECT
        Shipment_ID,
        COUNT(CASE
            WHEN Delay_Reason IS NOT NULL
            AND Delay_Reason != ''
            AND Delay_Reason != 'None'
            THEN 1
            ELSE NULL
        END) AS DelayCount
    FROM
        shipmenttrackingtable
    GROUP BY
        Shipment_ID)SELECT
    Shipment_ID,
    DelayCountFROM
DelayedCheckpointsWHERE
    DelayCount > 2 ORDER BY
    DelayCount DESC;
```

Result Grid | Filter Rows:

Shipment_ID	DelayCount

Video explaination

- https://drive.google.com/file/d/1jOPjPCXphlMWwEtH6MQ61OCbXos6qUuo/view?usp=sharing